ENGINEERING BUILD SPECIFICATION · HACKATHON EDITION

Nexora Knowledge Brain

An enterprise decision-intelligence system that doesn't just search company knowledge — it reasons across a live decision graph, enforces who is allowed to know what, and acts on it. This document is the complete design and build reference for the team.

Sprint: 15 days **Team:** backend + frontend **Stack:** Node · Neo4j · Groq · ChromaDB · Postgres · MCP

Status: design locked → build

---

§A What We're Building

Nexora Knowledge Brain is an internal AI assistant for a fictional enterprise SaaS company, Nexora Inc. Employees ask it questions in natural language; it answers from the company's collective knowledge — emails, meeting transcripts, Jira tickets, internal docs, and GitHub PRs — with a citation for every fact.

But it goes three steps beyond ordinary "AI search":

- **It reasons, not just retrieves.** It models company knowledge as a *graph of decisions* and walks that graph to answer "why" questions no keyword or vector search can.
- **It respects who you are.** Attribute-based access control means two people asking the same question get different answers based on their clearance — an intern literally cannot see what an executive can.
- **It acts.** It can file a real Jira ticket or draft a real email, not just talk about it.

§B Why It Matters

Every company loses knowledge. When someone leaves, the *why* behind decisions leaves with them. Reconstructing "why did we build it this way / why was this delayed / who approved that" costs senior-engineer-weeks. Existing tools (Glean, Notion AI) **retrieve documents** — they hand you the right file. They do not reconstruct the *story of why* across many connected sources, and their access control, while strong, is a means to an end rather than a visible, provable feature.

> **THE ONE-SENTENCE PITCH**
>
> Nexora doesn't just find the document — it reconstructs the *chain of decisions* behind any question, shows only what you're cleared to see, cites every claim, and takes the next action for you.

§C The Wow Moment

The demo is designed backward from one unforgettable moment: **watching the knowledge graph light up as the AI thinks.**

On screen: a dim map of the entire company's knowledge — every email, ticket, meeting, PR as a node. A judge asks *"Why was the Payments feature delayed?"* Then, in real time, the system **walks the graph**: a seed node ignites, then pulses outward along causal edges — `BLOCKS`, `CAUSED`, `EVIDENCES` — the reasoning path illuminating hop by hop. The answer streams in, and each citation **pulses its node** in the graph. The judge sees the machine *reason*, not just reply.

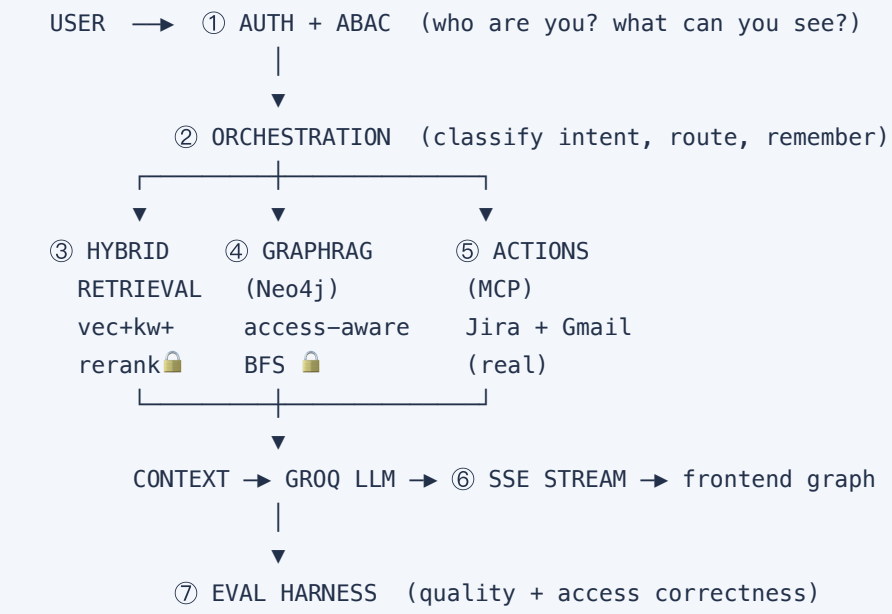
> **★ THE SECOND ACT — ACCESS CONTROL, MADE VISIBLE**
>
> Switch users live. The **CTO** asks about the security breach → the full chain lights up. The **intern** asks the same → the graph illuminates up to the security boundary and *goes dark exactly where their clearance ends.* Same question, same system — the access attributes alone produce two different lit subgraphs. Security you can *see*.

This moment is why every architectural decision below was made: the backend must traverse the graph *observably*, stream each step in real time, and enforce access *during* traversal.

30%	25%	25%	20%
Novelty GraphRAG + ABAC + MCP	Applicability real access ctrl + actions	Architecture 7 coherent layers	Docs this spec + evals report

§D Architecture Overview

Every query flows through seven layers. An orchestrator routes to three parallel capabilities; access control wraps retrieval and traversal; an eval harness verifies the whole thing.



---

01 Auth & ABAC NEW

Real JWT authentication. Users log in and receive a signed token carrying their *attributes*. We seed preset demo users (no signup flow) so identities can be switched instantly on stage.

WHAT	JWT login; each token carries `department`, `clearance`, `projects`.
WHY ABAC	Access based on *matching attributes*, not one role — so "an engineer *not* on the Payments project can't see Payments-confidential docs, even though they're a cleared engineer." That nuance is what real enterprise systems (and Glean) use.
WHY PRESET USERS	Instant identity-switching drives the wow moment's second act. A signup flow adds zero demo value.

---

02 Dataset REBUILD

~75 cross-referenced mock files across five source types, built as **four interconnected storylines**, each spanning all source types and carrying an access-sensitivity gradient.

STORYLINE	SPANS	ACCESS	ROLE
A · Payments delay	emails, tickets, meetings, PRs, docs	engineering, clr 2–3	the multi-hop "why" showpiece
B · Security breach 🔒	incident reports, confidential emails, postmortems	security/exec, clr 4–5, restricted	the ABAC drama — hidden from low clearance
C · DB migration + incident	tickets, emails, meetings, PRs	engineering, clr 2–3	connective tissue, ties A↔B
D · Search feature	all types	public, clr 1–2	the "everyone sees this" floor

FILE MIX 18 emails · 12 meetings · 20 tickets · 14 docs · 11 PRs = ~75.

WHY WOVEN The breach connects *into* the Payments thread (it exploited a rate-limiter vuln). So a low-clearance user traverses Payments freely, then hits a wall exactly where the breach begins — the restriction is part of the connected graph, not a separate silo.

ABAC TAGS Every file carries `access{department, projects, min_clearance, sensitivity}`.

---

03 Graph Schema — Neo4j ★ HEADLINE

The graph uses **two tiers** of nodes — this separation is what enables reasoning rather than mere lookup.

TIER 1 — ARTIFACTS (THE EVIDENCE)

`Email` · `Ticket` · `Meeting` · `PR` · `Doc` — each with an `access{}` property.

TIER 2 — CONCEPTS (THE MEANING)

`Person` · `Project` · `Decision` · `Incident` — the reasoning anchors that "why" questions resolve to.

RELATIONSHIPS

Structural: `REFERENCES`, `AUTHORED_BY`, `ABOUT`, `ATTENDED`. *Causal (the reasoning layer):* `CAUSED`, `BLOCKS`, `LED_TO`, `APPROVED`, `EVIDENCES`, `RESULTED_FROM`.

> **WHY TWO TIERS**
>
> A flat "documents-as-nodes" graph can only answer *what's related*. By making **Decisions** and **Incidents** first-class nodes with artifacts as *evidence*, "why" becomes a graph path — and when it lights up, the judge sees concepts surrounded by evidence, which *looks like reasoning*.

---

04 Concept Extraction NEW

How the Decision/Incident nodes and causal edges get built: **hybrid — authored backbone + LLM enrichment.**

BACKBONE	Core decisions, incidents, and causal edges are authored in the dataset — 100% reliable for the demo.
ENRICHMENT	At ingest, an LLM pass reads the docs and adds *additional* causal edges it discovers — the "the AI built its own graph" story.
WHY HYBRID	Reliability of a hand-built demo + the wow of AI-constructed connections. The backbone guarantees the demo never breaks; the LLM makes it feel alive.

---

05 Access-Aware Traversal NEW

Multi-hop reasoning uses **access-aware BFS**. The traversal checks every node's access at each hop; a restricted node is a *local wall* — the traversal skips it and keeps exploring every other route.

> **SECURE BY CONSTRUCTION**
>
> A path is valid only if **every node on it** passes the user's ABAC check (Cypher `ALL(n IN nodes(path) …)`). You cannot traverse *through* a restricted node to reach what's behind it — so downstream consequences of hidden information never leak. Nodes reachable by an *alternate* public path are still returned.

Visual payoff: the same query lights up different subgraphs per user; the graph goes dark precisely at each user's clearance boundary.

---

06 SSE Event Contract NEW · THE SPINE

The real-time event stream that drives the graph animation — the single most important contract between backend and frontend. The backend **paces** traversal events (~150–300ms apart) so the reasoning is *visible*.

EVENT	DRIVES
`auth_context`	"reasoning as: CTO" badge
`query_received`	detected intent
`graph_seed` / `node_activated` / `edge_traversed`	the graph lighting up, hop by hop
`node_blocked`	a wall flashes (carries *no* node data — can't leak)
`traversal_complete`	freeze the lit subgraph
`text`	answer tokens stream in
`citation_highlight`	citation → its node pulses
`tool_call` / `tool_result`	an action being taken + its result
`done` / `error`	close / graceful failure

A companion `GET /graph` endpoint returns the full access-filtered graph so the frontend renders the whole "company brain" as a dim backdrop, then lights the active path on top.

---

07 ABAC Policy Engine NEW

Declarative policy rules — access policies defined as *data* the engine evaluates, not hardcoded conditionals (the model AWS IAM / OPA use). One function, `canAccess(user, resource)`, is the single source of truth.

```
grant IF  user.clearance >= resource.min_clearance
      AND (resource.sensitivity == "public"
           OR user.department == resource.department
           OR user.department == "executive")
      AND (resource.projects is empty
           OR user.projects ∩ resource.projects ≠ ∅
           OR user.clearance >= 5)
```

The same `canAccess()` powers retrieval filtering, graph traversal, the `/graph` backdrop, and the eval harness — no duplicated logic.

---

08 Hybrid Retrieval IF TIME

Finds the right chunks before graph + LLM. Vector (semantic) + keyword (exact) + cross-encoder rerank.

```
query ─┬─▶ vector (ChromaDB)  top 15 ─┐
       └─▶ keyword (BM25)     top 15 ─┴▶ merge ▶ 🔒 ABAC filter ▶ Jina rerank ▶ top 8
```

WHY HYBRID	Vector search blurs exact IDs (`NEX-231` ≈ `NEX-204` to an embedding). Keyword nails them. You want both.
WHY FILTER MID-PIPELINE	Over-fetch, then drop restricted docs *before* rerank/LLM — so restricted data never reaches the model, and enough candidates survive.
RERANK	Jina's cross-encoder API — real reranking, no new infra (reuses the Jina key).

---

09 Orchestration & Memory NEW

ROUTING	An LLM intent classifier routes each query: "why/how" → GraphRAG; "draft/create/file" → MCP action; "summarize/who/what" → retrieval. LLM-based so it's robust to how judges phrase questions live.
MEMORY	Conversational memory (persisted, Postgres) so follow-ups resolve: "Who decided that?" → "Draft an email to them." Memory feeds routing.

10 MCP Actions NEW

The system *acts* via the Model Context Protocol. Actions are **real**: `create_ticket` → live Jira, `draft_email` → live Gmail draft. Internal tools: `extract_action_items`, `generate_report`.

> **DE-RISKING LIVE INTEGRATIONS (REQUIRED)**
>
> Every action tries the live API and **falls back to a convincing simulated result on any failure**, so the demo never breaks. Email creates a *draft* (not a send). OAuth tokens are pre-authorized and dry-run before the demo. Every action is logged to the Postgres audit log and gated by ABAC.

11 Eval Harness NEW

An automated suite that runs the demo questions and **scores** the system — proving it works rather than claiming it. Measures answer correctness, citation accuracy, groundedness, refusal honesty, and — uniquely — **access correctness**.

> **THE DIFFERENTIATOR: ACCESS-CORRECTNESS**
>
> Runs the *same question as different users* and verifies the intern is denied what the CTO can see. Output: an evals report with a line like `100% access-control compliance` — a pitch mic-drop, and exactly what a real security team would demand.

12 Postgres Backbone NEW

A real relational database anchoring four things — so it earns its place as more than a chat store:

- **Users + ABAC attributes** — the auth store
- **Conversation history** — persisted memory
- **Audit log** — every query, who asked, what access was granted/denied (the enterprise-compliance story)
- **Action records** — every MCP action taken

13 The Three Enhancements NEW

Thin touches that ride on existing layers — sparkle, no new infrastructure.

- **Conversational memory** → multi-turn follow-ups (lives in orchestration).
- **Graph-linked citations** → each citation expands to its source + graph connections; pulses its node as the answer streams (bridges GraphRAG to the answer).
- **Honest "I don't know"** → refuses when the knowledge base lacks the answer, surfaced as a trust feature.

14 The Demo ⭐

Five questions, designed to exercise every capability and build to the ABAC climax:

#	QUESTION	SHOWS
1	Why was the Payments feature delayed?	multi-hop causal — the graph lights up
2	Who approved the database migration?	factual + citations
3	Summarize the security incident.	🔒 CTO gets full breach; intern gets "restricted" — the ABAC moment
4	Draft a follow-up email about the rate-limiting issue.	real MCP action (Gmail draft)
5	What chain of events led to the breach?	🔒 GraphRAG + ABAC combined, exec-only deep traversal

§E File-by-File Structure

Every file and its single responsibility — the build map.

```
nexora-knowledge-brain/
├── server/
│   ├── config/
│   │   ├── env.js          # load + validate env, fail fast
│   │   ├── groq.js         # Groq client + model constant
│   │   ├── chroma.js       # ChromaDB client + 'knowledge' collection
│   │   ├── neo4j.js        # Neo4j driver + session factory
│   │   ├── postgres.js     # PG pool + schema init
│   │   ├── embeddings.js   # swappable embedder (Ollama|Jina)
│   │   └── llamaindex.js   # chunking (SentenceSplitter)
│   ├── auth/
│   │   ├── jwt.js          # sign / verify tokens
│   │   ├── middleware.js   # extract user + attrs from token
│   │   ├── policy.js       # canAccess() + declarative rules
│   │   ├── attributes.js   # attribute shapes + validators
│   │   └── users.js        # preset demo users → Postgres
│   ├── ingestion/
│   │   ├── scanner.js      # recursive data/ scan
│   │   ├── readers.js      # file → document + ABAC metadata
│   │   └── loader.js       # scan→read→chunk→embed→store
│   ├── graph/
│   │   ├── schema.js       # node labels, relations, shapes
│   │   ├── extractor.js    # authored refs + LLM causal edges
│   │   ├── builder.js      # write nodes/edges to Neo4j
│   │   └── traversal.js    # access-aware BFS + step events ⭐
│   ├── retrieval/
│   │   ├── vector.js       # ChromaDB semantic search
│   │   ├── keyword.js      # BM25 search
│   │   ├── rerank.js       # Jina cross-encoder rerank
```

```
│   │   └── hybrid.js       # merge + ABAC filter + rerank
│   ├── agent/
│   │   ├── orchestrator.js # route + memory + assemble + stream
│   │   ├── intent.js       # LLM intent classifier
│   │   ├── memory.js       # conversation memory (Postgres)
│   │   ├── prompts.js      # SYSTEM_PROMPT + context builders
│   │   └── stream.js       # SSE event emitter (14-event contract) ★
│   ├── mcp/
│   │   ├── client.js       # MCP client
│   │   ├── tools.js        # tool schemas
│   │   ├── internal.js     # extract_action_items, generate_report
│   │   └── servers/
│   │       ├── jira.js     # Jira MCP server (real + fallback)
│   │       └── gmail.js    # Gmail MCP server (real + fallback)
│   ├── evals/
│   │   ├── questions.js    # suite: question, user, expected + access
│   │   ├── run.js          # run suite through real system
│   │   ├── judge.js        # LLM-as-judge scoring
│   │   └── report.js       # scored report table
│   ├── routes/
│   │   ├── auth.js         # POST /login → JWT
│   │   ├── query.js        # POST /query → SSE stream ★
│   │   ├── graph.js        # GET /graph → filtered backdrop
│   │   ├── ingest.js       # POST /ingest → full pipeline
│   │   ├── sources.js      # GET /sources → doc counts
│   │   └── audit.js        # GET /audit → audit log
│   ├── middleware/
│   │   ├── errorHandler.js  rateLimiter.js  auditLogger.js
│   ├── utils/logger.js
│   └── index.js           # wire middleware, routes, startup
├── data/                  # ~75 mock files
├── docker-compose.yml     # Neo4j + Postgres + ChromaDB
├── .env.example    package.json    README.md
```

§F Setup & Run

```
# 1. Infra — one command (Neo4j + Postgres + ChromaDB)
docker compose up -d

# 2. Embeddings — Ollama local, or Jina cloud fallback
ollama serve &  ollama pull all-minilm

# 3. Install + ingest (builds vectors AND the graph)
npm install
npm run ingest

# 4. Run
npm start           # API on :3001

# 5. Prove it works
npm run evals       # generates the scored evals report
```

ENVIRONMENT

`GROQ_API_KEY` · `JINA_API_KEY` · `NEO4J_URI/USER/PASSWORD` · `POSTGRES_URL` · `CHROMA_URL` · `OLLAMA_URL` · `EMBEDDING_PROVIDER` · `JWT_SECRET` · `JIRA_*` · `GMAIL_*` · `PORT=3001`